

EXPERIMENT-8

Student Name: Husanpreet Kaur
Branch: AIT-CSE (Cyber Security)
Semester: 5
Subject Name: Computer Networks

UID: 24BCY70138
Section/Group: 24AIT_KRG-G2
Date: 16/09/2026
Subject Code: 24CSH-337

AIM: To implement a client-server communication system using Python TCP sockets over the IPv4 loopback interface, and to demonstrate connection establishment, message transmission, server responses, and graceful connection termination.

OBJECTIVES:

- To understand the basic concept of socket programming.
- To understand the client-server communication model.
- To understand the difference between TCP and connectionless communication.
- To create an IPv4 TCP socket using Python's socket module.
- To configure a server socket using bind() and listen().
- To accept a client connection using accept().
- To connect a client to the server using connect().
- To transmit data between client and server using send() and recv().
- To understand encoding and decoding of messages during socket communication.
- To use the localhost address 127.0.0.1 and TCP port 5000 for local communication.
- To implement a simple request-response communication loop.
- To terminate the TCP connection correctly using a predefined message and close().

SOFTWARE REQUIREMENTS:

(a) Software:

- Python 3.x
- Visual Studio Code
- Windows Operating System
- Python socket module (built-in; no external package required)

EXPERIMENT-8

(b) Hardware (Virtual):

- One computer/laptop
- Network interface with TCP/IP support
- Local loopback interface

THEORY: (i) What is Socket Programming?

Socket programming is a method of enabling two programs to communicate over a network. A socket acts as an endpoint of communication. In a client-server model, the server creates a socket and waits for incoming connections, while the client creates a socket and connects to the server.

(ii) What is a Socket?

A socket is identified by an IP address, a port number, and a transport protocol. In this experiment, Python creates an IPv4 TCP socket using `AF_INET` and `SOCK_STREAM`.

Parameter	Meaning
<code>AF_INET</code>	Uses IPv4 addressing.
<code>SOCK_STREAM</code>	Creates a stream-oriented TCP socket.
<code>127.0.0.1</code>	IPv4 loopback/localhost address used for communication on the same computer.
<code>5000</code>	TCP port on which the server listens.

(iii) What is TCP?

TCP (Transmission Control Protocol) is a connection-oriented transport-layer protocol. Before application data is exchanged, a TCP connection is established between the client and server. TCP provides reliable, ordered, and error-checked delivery of a byte stream.

(iv) Client-Server Model:

EXPERIMENT-8

The server starts first and waits for a client. The client then establishes a TCP connection with the server. After connection establishment, both programs can exchange application data.

Component	Main Responsibility
Server	Creates socket, binds IP/port, listens, accepts a client, receives messages and sends replies.
Client	Creates socket, connects to the server, sends messages and receives replies.

(v) Important Socket Functions:

Function	Purpose
socket()	Creates a new socket.
bind()	Associates the server socket with an IP address and port.
listen()	Places the server socket into a listening state.
accept()	Accepts an incoming client connection.
connect()	Establishes a client connection to the server.
send()	Sends bytes through the connected socket.
recv()	Receives bytes from the connected socket.
close()	Closes the socket and releases the connection.

(vi) IPv4 Loopback Address:

The address 127.0.0.1 is the IPv4 loopback address. It refers to the same computer on which the program is running. Therefore, this experiment does not require two physical computers. Both the server and client can run on the same machine.

(vii) Port Number:

EXPERIMENT-8

A port identifies a particular application endpoint on a host. In this experiment, TCP port 5000 is used by the server. The client connects to 127.0.0.1:5000.

(viii) Encoding and Decoding:

Socket send and receive operations work with bytes. The client converts a Python string into bytes using `encode()` before sending it. The receiver converts received bytes back into a string using `decode()`.

Eg:- `message.encode()` converts a string into bytes

And `data.decode()` converts bytes back to a string.

(ix) TCP Connection and Data Flow:

The communication sequence used in this experiment is:

1. Server creates an IPv4 TCP socket.
2. Server binds the socket to 127.0.0.1 and port 5000.
3. Server starts listening for a connection.
4. Client creates an IPv4 TCP socket and connects to 127.0.0.1:5000.
5. Server accepts the connection.
6. Client sends a message.
7. Server receives the message, displays it, and sends a reply.
8. The exchange continues until the client sends the message "Over".
9. Both sides close their sockets.

STEPS/IMPLEMENTATION:

Part I – Prepare the Project:

1. Open Visual Studio Code.
2. Open the TCP_Socket_Experiment project folder.

EXPERIMENT-8

3. Ensure that server.py and client.py are present in the project folder.
4. Python 3.x should be installed and available in the VS Code terminal.

Part II – Server Program:

1. Open server.py.
2. Import the socket module.
3. Create an IPv4 TCP socket.
4. Bind the socket to 127.0.0.1 and port 5000.
5. Call listen(1) to wait for an incoming client.
6. Call accept() to establish the connection.
7. Receive messages using recv(1024).
8. Decode received bytes and display the client message.
9. Send a reply using encode() and send().
10. Stop the loop when the client sends "Over" and close the sockets.

--- server.py---

```
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(("127.0.0.1", 5000))
server_socket.listen(1)
print("Server started")
print("Waiting for a client...")
client_socket, client_address = server_socket.accept()
print("Client connected:", client_address)
while True:
```

EXPERIMENT-8

```
message = client_socket.recv(1024).decode()
print("Client:", message)
if message == "Over":
    break
reply = input("Enter reply: ")
client_socket.send(reply.encode())
client_socket.close()
server_socket.close()
print("Connection closed")
```

Part III- Client Program:

1. Open client.py.
2. Import the socket module.
3. Create an IPv4 TCP socket.
4. Connect to the server at 127.0.0.1:5000.
5. Read a message from the user.
6. Encode the message and send it to the server.
7. Receive and display the server's reply.
8. If the message is "Over", terminate the communication loop.
9. Close the client socket.

---client.py---

```
import socket
```

```
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

EXPERIMENT-8

```
client_socket.connect(("127.0.0.1", 5000))
print("Connected to server")
while True:
    message = input("Enter message: ")
    client_socket.send(message.encode())
    if message == "Over":
        break
    reply = client_socket.recv(1024).decode()
    print("Server:", reply)
client_socket.close()
print("Connection closed")
```

PROCEDURE:

1. Network Setup:

- Open Visual Studio Code.
- Open the TCP_Socket_Experiment project.
- Verify that server.py and client.py are available.
- Open the integrated terminal.

2. Server Configuration:

- Create an IPv4 TCP socket.
- Bind the socket to 127.0.0.1:5000.
- Put the server into listening mode.
- Accept an incoming client connection.

3. Client Configuration:

- Create an IPv4 TCP socket.
- Connect the client to 127.0.0.1:5000.

EXPERIMENT-8

- Verify successful connection establishment.

4. Data Communication:

- Enter a message on the client.
- Send the message to the server.
- Receive the message at the server.
- Enter a reply at the server.
- Send the reply to the client.
- Display the received reply.

5. Connection Termination:

- Enter "Over" on the client.
- Stop the communication loop.
- Close the client socket.
- Close the server socket.

INPUT/OUTPUT DETAILS:

(a) Communication Parameters:

Parameter	Value
Communication Model	Client-Server
Transport Protocol	TCP
Address Family	IPv4
Server IP	127.0.0.1
Server Port	5000
Client	client.py
Server	server.py
Buffer Size	1024 bytes

EXPERIMENT-8

Termination Message	Over
---------------------	------

(b) Sample Message Exchange:

Step	Client	Server
1	Connects to 127.0.0.1:5000	Accepts client connection
2	Hello Server	Receives: Hello Server
3	Waits for reply	Sends: Hello Client
4	Displays: Server: Hello Client	Waits for next message
5	Over	Receives Over and terminates

COMMANDS:

```
socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
server_socket.bind(("127.0.0.1", 5000))
```

```
server_socket.listen(1)
```

```
server_socket.accept()
```

```
client_socket.connect(("127.0.0.1", 5000))
```

```
client_socket.send(message.encode())
```

```
client_socket.recv(1024).decode()
```

```
client_socket.close()
```

DEMONSTRATION/OUTPUT:

EXPERIMENT-8

```
● PS E:\Download Docs\UNIT-2-Chapter 2.2 - CN.pptx  
  (1)\TCP_Socket_Experiment> cd TCP_Socket_Experi  
ment  
○ PS E:\Download Docs\UNIT-2-Chapter 2.2 - CN.pptx  
  (1)\TCP_Socket_Experiment\TCP_Socket_Experiment  
> python server.py  
Server started  
Waiting for a client...  
Client connected: ('127.0.0.1', 43422)  
Client: hello  
Enter reply: hi'  
█
```

```
○ PS E:\Download Docs\UNIT-2-Chapter 2.2 - CN.pptx  
  (1)\TCP_Socket_Experiment\TCP_Socket_Experiment  
> python client.py  
Connected to server  
Enter message: hello  
Server: hi'  
Enter message: █
```

```
Help  TCP_Socket_Experiment  
client.py X  
TCP_Socket_Experiment > client.py > ...  
1  
2 import socket  
3  
4 client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
5  
6 ## SOCK_STREAM: represents TCP Segment / TCP Data  
7  
8 client_socket.connect(("127.0.0.1", 5000))  
9  
10 ## Client connecting to server with 127.0.0.1 on port 5000  
11  
12 print("Connected to server")  
13  
14 while True:  
15     message = input("Enter message: ")  
16  
17     client_socket.send(message.encode()) ##Encode(): Converts raw string to bytes  
18  
19     if message == "Over":  
20         break  
21  
22     reply = client_socket.recv(1024).decode()  
23  
24     print("Server:", reply)  
25  
26 client_socket.close()  
27  
28 print("Connection closed")
```

EXPERIMENT-8

```
server.py X
CP_Socket_Experiment > server.py > ...
1 import socket
2 ##This module lets programs communicate over a network.
3
4 server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5
6 ##socket.AF_INET: means use IPV4
7 ##socket.SOCK_STREAM: means use TCP,
8 ##In short: Create an IPV4 TCP Socket.
9
10 server_socket.bind(("127.0.0.1", 5000)) ##Binding Server IP and Port
11 ##127.0.0.1 means localhost, so both program will run on same PC.
12
13 server_socket.listen(1)
14 ##Server, start waiting for incoming TCP connection requests.
15
16 print("Server started")
17 print("Waiting for a client...")
18
19 client_socket, client_address = server_socket.accept()
20 ## Accept waits until a client establishes a TCP Connection.
21 ## If no client has connected, the program will pause over here only.
22
23 ''' This command waits for a client.
24 The server stops here until a client connects.'''
25
26
27 print("Client connected:", client_address)
28
29 while True:
30     message = client_socket.recv(1024).decode()
31     ## Will decode the coming message from the client of 1024 bytes.
32     # It attempts to receive up to 1024 bytes from the TCP connection.
```

RESULT: The experiment was successfully implemented using Python TCP socket programming. An IPv4 TCP server was created and bound to 127.0.0.1:5000, and a client successfully established a TCP connection with the server.

Messages were transmitted from the client to the server and replies were returned using send() and recv(). String encoding and decoding were used for byte-based socket communication. The session was terminated successfully using the "Over" message, followed by proper socket closure.

LEARNING OUTCOMES:

EXPERIMENT-8

After completing this experiment:

- I understood the client-server model and the role of sockets in network communication.
- I learned to create and configure IPv4 TCP sockets using Python's socket module.
- I learned the purpose of bind(), listen(), accept(), connect(), send(), recv(), and close().
- understood how messages are encoded into bytes for transmission and decoded after reception.
- I developed practical skills in running and testing a TCP client-server application using localhost and a TCP port.